

Response to Reviewers

GaugeFixer: overcoming parameter non-identifiability in models of
sequence-function relationships

Carlos Martí-Gómez, David M. McCandlish, Justin B. Kinney

Submission ID: 9a7ff349-d9aa-48f2-8210-3e1125bbde71

7 August 2026

We thank the Editor and the two Reviewers for their consideration of our manuscript. This work has been substantially improved as a result of the feedback we recieved. Additional edits have been made throughout the manuscript to improve clarity. Below is a point-by-point response to each of the critiques raised.

We also note that, since submitting the initial manuscript, we realized that several methods proposed by other groups for analyzing sequence-function relationships can also be computed efficiently using the algorithms we describe (Weinberger, 1991; Poelwijk et al., 2016; Stormo, 2011; Brookes et al., 2022; Tsui et al., 2026; Faure et al., 2024). We have implemented these methods in GaugeFixer, and have added a paragraph to the Discussion explaining how our algorithms accelerate these computations and extend them to hierarchical models.

Finally, Algorithms 1 and 2 can also be used for purposes other than gauge fixing. In particular, the Walsh-Hadamard transformation of sequence-function landscapes (Weinberger, 1991; Poelwijk et al., 2016) and its generalizations (Stormo, 2011; Brookes et al., 2022; Tsui et al., 2026; Faure et al., 2024) involve multiplying an α^L -dimensional vector (representing a combinatorially complete landscape) by a dense matrix constructed as a Kronecker product of position-specific matrices. Algorithm 1 allows these transformations to be performed far more efficiently than by standard matrix multiplication. Algorithm 2, moreover, extends these transformations to landscapes that are defined by hierarchical models (e.g., K -order or K -adjacent models), rather than by combinatorially complete sets of measurements. This latter capability should allow these methods to be applied to sequences that are much longer than is computationally feasible using published approaches. We have incorporated these other landscape-analysis approaches into GaugeFixer alongside the package’s core gauge-fixing functionality.

Editor Comments

Thank you for submitting your manuscript to BMC Bioinformatics. Please see below for the comments from the reviewers. The reviewers have raised points that require minor revision of the manuscript before it is approved for the publication. Please address their comments carefully.

Dongheon Lee

Please address the points mentioned below along with the editorial comments.

1. Please ensure that the statement given in the “Data Availability” section of the SNAPP submission system matches the statement included in the “Availability of Data and Materials” section of the manuscript. When submitting your revised manuscript, please select ‘yes’ under the data availability declaration and input your data availability statement in the appearing textbox.

We recommend submitting all revisions within the mentioned deadline.

If you need more time, please contact us and include your submission ID.

Kind regards,

Payel Karmakar
Assistant Editor
BMC Bioinformatics

Support contact: bmcbioinformatics@biomedcentral.com

We have selected “yes” under the data availability declaration, and have entered the manuscript’s “Availability of data and materials” section into the SNAPP submission system.

Reviewer 1

In this paper, building on previous work of a more theoretical nature, the authors define and illustrate a practical method for fixing the “gauge symmetries” that hamper the interpretability of even the most basic sequence-to-phenotype models. This is an important topic that is not on most people’s radar screens: two equivalent models can have very different parameters even if their predictions are identical. The authors include a software package that allows users to standardize the parameters of their own models. This is a very useful contribution to the field in my opinion.

I have one suggestion: the fact that the boundary case at position -9 is distinct from more upstream positions could have two explanations: (i) an intrinsically positional effect or (ii) the pentamer window used is not wide enough to fully capture the sequence-to-phenotype relationship, which, if the fixed base at position -4 in the library design is suboptimal, would partly explain the steep drop in panel 3B. This would be easy to check explicitly, and would add to the analysis.

We thank the reviewer for this suggestion. To distinguish these two explanations, we repeated the analysis using hierarchical gauges defined by the extended motifs GAGGAG and AGGAGU, which place the fixed $-14G$ and $-4U$ of the experimental construct immediately adjacent to the core motif. The results support the reviewer’s hypothesis in part: the fixed $-4U$ does appear to contribute to the low mean activity of sequences carrying the core motif at register -9 , although this contribution is itself register-dependent. The fixed $-14G$, by contrast, does not account for the reduced activity at register -13 . We have updated Figure 3B to include these results and have added the following text to the Results section:

The dependence of the constant parameter on register could reflect the intrinsic positional preference of the core motif, the influence of the fixed sequence flanking the variable region, or both. Every assayed sequence carries a fixed G at position -14 and a fixed U at position -4 , just outside the variable region. The effects of these characters cannot be read directly from the model. They can, however, be probed indirectly: extending the core motif to GAGGAG or AGGAGU places a G or a U immediately adjacent to the core, and the resulting constant parameter reports the mean activity of sequences carrying that character at that position. We therefore computed constant parameters under hierarchical gauges defined by these extended motifs at every register where they fit within the variable region (Figure 3B). Neither extended motif can be placed at the register where its added character would coincide with the corresponding fixed position in the assay: AGGAGU does not fit at register -9 , nor GAGGAG at register -13 . The

neighboring registers, -10 and -12 respectively, therefore provide the closest available proxies. At register -10 , the mean activity of sequences containing AGGAGU was substantially lower than that of sequences containing AGGAG alone; a smaller decrease was observed at register -13 , and essentially none at registers -12 and -11 . This suggests that the low average activity of sequences carrying the core motif at register -9 is partly attributable to the negative contribution of the fixed $-4U$, although this contribution is itself register-dependent. By contrast, the extended motif GAGGAG raised mean activity at registers -11 through -9 , but not at register -12 . The fixed $-14G$ is therefore unlikely to explain why the constant parameter is lower at register -13 than at registers -12 and -11 ; an additional upstream G appears to be beneficial only when the core motif lies closer to the start codon.

Minor comments:

1. Abstract, “scales quadratically”: This is in terms of “ M ”, which itself is defined as $(1 + \alpha)^L$, so exponential in the alphabet size. It would be good to clarify this.

The revised abstract clarifies that the size of the projection matrix scales quadratically with the number of model parameters.

The most direct computational implementation of these methods is often impractical, however, because it requires a projection matrix whose size scales quadratically with the number of model parameters.

The number of parameters is the relevant quantity here, because its scaling with sequence length and alphabet size depends on the specific model considered. Although the total number of parameters is $(\alpha + 1)^L$ for an all-order model, this expression does not apply to the more general class of hierarchical models. For example, the number of parameters scales linearly with L in an additive model and quadratically with L in a pairwise-interaction model. We have revised the Technical Background section to clarify this distinction:

It is therefore common to use models that include only interactions up to a specified order (K -order models) or only interactions between nearby positions (K -adjacent models). The number of parameters in these models grows only polynomially with sequence length (as L^K for K -order models and linearly in L for K -adjacent models), making them tractable for much longer sequences.

2. Figure 1: it is a little unclear to me how the lines runs relative to the blue plane. Perhaps choosing a different 3D viewpoint would help, and perhaps other improvements can be made to make this important figure as clear as possible.

We have reworked Figure 1 to more clearly show the positions of the parameter choices, the zero-sum gauge plane, and the line of equivalent parameters in 3D space.

3. Page 3, left column, 2nd paragraph, “restricted to subsets of positions”: Can the authors clarify whether these subsets can be overlapping or not?

The subsets of positions used in the decomposition (the generating sets) may overlap, and generally do so except in special cases such as additive models or models defined by disjoint generating

sets. Such overlaps affect neither the applicability nor the correctness of the algorithm. We have clarified this point in the text by adding:

Note that there are often many valid ways to carry out this decomposition due to the generating sets in W overlapping. Because P is a linear operator, however, the result is independent of which decomposition is used.

4. Near Equation (4) the authors also use the terms “feature” and “subset of positions” interchangeably. Good to explicitly say that they are the same thing,

These two terms are actually intended to represent different objects: a feature x_U^u is defined by a subset of positions U and a subsequence u of length $|U|$. It takes value 1 if the subsequence at positions U for sequence s matches u exactly, and 0 otherwise. We have rewritten this passage to make the distinction explicit:

Formally, each feature $x_U^u(s)$ is specified by a subset of positions $U \subseteq S$ and a subsequence $u \in \mathcal{A}^{|U|}$:

$$x_U^u(s) = \begin{cases} 1 & \text{if } s[U] = u \\ 0 & \text{otherwise} \end{cases},$$

where $s[U]$ indicates the subsequence of s that occurs at the positions U .

5. Under Equation 9: Make explicit that since $M = (1 + \alpha)^L$, scaling as $O(L(1 + \alpha)M)$ is the same as $O(L(1 + \alpha)^{1+L})$, etc.

We have substantially clarified the scaling relations in the revised manuscript. For all-order models, $M = (1 + \alpha)^L$, so $L = \log_{1+\alpha} M$. Therefore, for a fixed alphabet size α , the computational complexity $O(L(1 + \alpha)M)$ is actually equivalent to $O(M \log M)$, not $O(M)$ as we previously stated. We have revised the text to make this equivalence explicit and to state the corresponding comparison with direct matrix multiplication:

For each choice of l and indices $i_1, \dots, i_L$, it takes $\alpha + 1$ multiplication-addition operations to compute the right-hand side of Eq. 10. As there are L values of l and $(\alpha + 1)^L$ possible combinations of indices, the algorithm has computational complexity $O(L(\alpha + 1)^{L+1})$. In terms of M and α , this is $O(\frac{\alpha+1}{\log(\alpha+1)} M \log M)$, which for fixed α becomes $O(M \log M)$. By contrast, standard matrix multiplication by P requires $O(M^2)$ operations.

The memory requirements are treated in the following paragraph:

Since $M \geq (\alpha + 1)^2$ whenever $L \geq 2$, and the algorithm never stores more than two M -sized vectors at once, these requirements scale as $O(M)$. By contrast, storing P itself requires storing $O(M^2)$ numbers in memory.

We have also added the computational and memory complexities of Algorithm 2:

The memory requirements of this algorithm scale as $O(M)$, just as for Algorithm 1. The computational requirements, however, depend on the size of W and on the size of each $S' \in W$, both of which are model-specific. For K -order models specifically, the generating sets consist of all possible subsets of K positions; this means that $|W| = \binom{L}{K}$ and $|S'| = K$ for all $S' \in W$. The primary computational expense

occurs in line 5, which is carried out $\binom{L}{K}$ times. Each execution requires $K(\alpha + 1)^{K+1}$ operations, following the analysis above. By comparison, the number of parameters is $M = \sum_{k=0}^K \binom{L}{k} \alpha^k$. Assuming $K \ll L$, we can approximate $M \approx \binom{L}{K} \alpha^K$. Holding α and K fixed while increasing L thus causes the computational complexity to scale as $O(M)$. K -adjacent models behave similarly.

6. Figure 3C: Consider moving label “register −13” etc from left of heat map to above the heat map, or even in between rows C and D because it applies to both.

The register labels were intended to apply jointly to the two aligned heatmaps, but we agree that this was not obvious. To make this clearer, we have relabeled the former panels C and D as the left and right parts of a single panel C, representing the additive and pairwise parameters for each register, respectively. We have also adjusted the register labels to clarify that each applies to both the rectangular and triangular heatmaps in that row.

Reviewer 2

I find this work of high quality and broad interest and I have only minor remarks.

1. Introduction: Sentence ‘This non-identifiability occurs because the binary feature vectors representing the sequences do not span the full space in which they live’ – could the authors check again this statement, or unpack it? I am not sure it’s true, in the sense that even if I had an exhaustive dataset the issue has to do with statistical observables (i.e. frequencies) not providing the same amount of constraints as the parameters to infer, as explained at page 5 here: <https://arxiv.org/pdf/1703.01222>

Having fewer independent observables than raw parameters is another way of describing the same underlying redundancy: generalized one-hot models are overparametrized, so multiple parameter vectors can represent exactly the same sequence-function map. Importantly, this is a structural property that results from the mathematical form of the model, rather than a consequence of the data used for inference. It therefore applies to any sequence-function model expressed in this form, including cases in which the function value is known exactly for every possible sequence. We have revised the text to make this point and the corresponding geometry more explicit.

A fundamental challenge in interpreting the parameters of these models is that, except in trivial cases, parameter values are not uniquely determined by the sequence-function landscape that the model describes (Posfai et al., 2025a). This structural non-identifiability is a consequence of the model’s mathematical form and is independent of the type and amount of data used to fit it. It occurs because the binary feature vectors representing all possible sequences span only a proper subspace of the vector space in which they are defined. Consequently, a continuum of parameter vectors yields identical model predictions. These parameter vectors differ from one another along what are called “gauge freedoms”—directions in parameter space that are orthogonal to the binary feature vectors of all possible sequences.

2. page 3: ‘In this case there are $M = (\alpha + 1)^L$ parameters’ – just for clarity, could the author

explain why $\alpha + 1$ per position, looping back to the minimal example? (I guess the ‘+1’ is the offset θ_0 there).

Summing over all subsets of positions directly gives $M = \sum_{k=0}^L \binom{L}{k} \alpha^k = (\alpha + 1)^L$. Alternatively, we show that each parameter can be indexed by a sequence of length L constructed from an extended alphabet of $\alpha + 1$ characters, the extra character being a wild-card ‘*’ marking positions not included in U . This latter rationale is actually the basis for the indexing scheme used by the projection matrices.

*An all-order model has $M = (\alpha + 1)^L$ parameters. To see this, note that there are $\binom{L}{k}$ ways to choose a set of positions U of size k , and α^k possible subsequences u for each such set. Summing over k gives $\sum_{k=0}^L \binom{L}{k} \alpha^k = (\alpha + 1)^L$ parameters. Alternatively, we can derive this parameter count by assigning to each feature x_U^u an index sequence of length L that has subsequence u at the positions in U and a wild-card character * at all other positions. Because these index sequences are constructed from the alphabet $\{*, c_1, \dots, c_\alpha\}$, which has size $\alpha + 1$, there are $(\alpha + 1)^L$ possible index sequences, and hence the same number of parameters. This indexing approach underlies the tensor representation introduced by Posfai et al. (2025b) and used below.*

3. Eqs 8-9: I would clarify the range of variation of $i_1, \dots, i_L$, and $j_1, \dots, j_L$

We have rewritten these equations (now Eqs. 9 and 10) to show explicitly the range of summation for each index, as well as the fact that all possible combinations of indices are summed over:

$$[\vec{\theta}_{fixed}]_{i_1 \dots i_L} = \sum_{j_1=1}^{\alpha+1} \sum_{j_2=1}^{\alpha+1} \dots \sum_{j_L=1}^{\alpha+1} [P_1]_{i_1 j_1} \dots [P_L]_{i_L j_L} [\vec{\theta}]_{j_1 \dots j_L},$$

where each index i_l runs from 1 to $\alpha + 1$.

4. ‘To project a hierarchical model into a hierarchical gauge’ $\rightarrow$ ‘onto’?

We have rewritten this sentence, which now reads:

To express the parameters of a hierarchical model in a hierarchical gauge, we use the fact that these parameters can be decomposed into a sum of the parameters of multiple all-order models, each restricted to one of the generating position sets.

5. Page 4: around algorithm 2, I was a bit confused as to why in some instances one has $\vec{\theta}[S']$ and in others $\vec{\theta}_{S'}[S']$, do they mean the same thing? Can one homogenize or clarify the notation?

We apologize for the confusion. $\vec{\theta}_{S'}[S']$ and $\vec{\theta}_{S'}$ convey the same information but differ in their dimensions: $\vec{\theta}_{S'}$ has $(\alpha + 1)^L$ entries, of which only $(\alpha + 1)^{|S'|}$ can be nonzero; $\vec{\theta}_{S'}[S']$ is the $(\alpha + 1)^{|S'|}$ -dimensional vector restricted to those nonzero entries. We agree that this notation is cryptic, and have therefore clarified Algorithm 2 by introducing an intermediate variable $\vec{\omega} \leftarrow \vec{\theta}_{\text{tmp}}[S']$. We have also clarified the meaning of the bracket notation:

Here, $\vec{\theta}[S']$ indicates an $(\alpha + 1)^{|S'|}$ -dimensional vector composed only of the elements θ_U^u for which $U \subseteq S'$, and $\pi[S']$ denotes the marginal distribution over subsequences on S' .

6. Could the authors clarify what ‘register’ means in this ribosome-binding context?

We have added a clause defining the concept of register:

Doing so, they observed that the SD landscape contains multiple prominent fitness peaks corresponding to the canonical AGGAG motif appearing in different registers (i.e., starting at different positions upstream of the start codon).

The following paragraph also carries a footnote stating the indexing convention used throughout this section:

[FOOTNOTE: Throughout this section we index sequence positions l by their offset from the start codon, so that l runs from -13 to -5 rather than from 1 to 9 . We also denote by i the register of the AGGAG core motif, so that i runs from -13 to -9 .]

7. Discussion: ‘it can also be applied to nonlinear and nonparametric models, such as neural networks or Gaussian processes. This is done by representing the predicted landscape using an all-order model and applying GaugeFixer to its parameters.’ – Could the authors unpack this? Does it mean to locally approximate the deep-learning landscape by an all-order model, a bit like LIME does for linear regression?

This procedure does not involve any local approximation, such as that used by LIME. Rather, one first evaluates the nonlinear or nonparametric model on every possible sequence. One then expresses the resulting landscape as an all-order model and uses GaugeFixer to transform its parameters into the desired gauge. We have revised Results to clarify this point:

Here we used GaugeFixer to characterize the landscape around each fitness peak. We represented the combinatorially complete landscape of Martí-Gómez et al. (2026) as an all-order model having $(4 + 1)^9 = 1,953,125$ parameters expressed in the trivial gauge (in which only the 4^9 parameters of order 9 are nonzero).

We have revised the Discussion to emphasize this point:

Although GaugeFixer operates only on specific classes of linear models (all-order and hierarchical models), it can still be used to characterize nonparametric or nonlinear models (e.g., Gaussian process models or deep neural networks). To do this, one first evaluates the model of interest on every possible sequence. The resulting landscape is then represented as an all-order model, and GaugeFixer is applied to the corresponding parameters. This is what we did for the Shine-Dalgarno landscape analyzed above (Figure 3). Because this approach requires explicitly representing the combinatorially complete landscape, it can be used only when sequences are short enough for that to be computationally feasible. We note that recent theoretical results, however, suggest a way of overcoming this limitation for certain Gaussian process models (Petti et al., 2026).

8. Figure 2: could the authors clarify: if a time limit was set for standard gauge fixing that explains the curves stopping at 10^4 parameters, and which one; the characteristics of the model (α , L , etc.)

No time limit was imposed on the standard gauge-fixing procedure. Instead, the benchmark was limited by the memory required to construct the dense projection matrix. Peak memory

reached 1.4 GB for the largest all-order model tested (9,261 parameters) and 0.6 GB for the largest pairwise model tested (6,121 parameters). Because this requirement grows as M^2 , the next all-order model in the series (194,481 parameters) would have required roughly 600 GB, and the next pairwise model (11,361 parameters) roughly 2 GB. We therefore restricted the benchmark to tasks requiring less than 2 GB of memory. We have also stated the range of sequence lengths tested for each model class. The revised Figure 2 legend reads:

Benchmarking results. Shown are the runtime (A) and peak memory (B) requirements for GaugeFixer and for standard matrix multiplication, plotted as a function of the number of model parameters. Tests were carried out for all-order models ($L = 2$ to 5) and pairwise models ($L = 3$ to 150) on protein sequences ($\alpha = 20$). Each data point represents the mean and standard deviation over 10 gauge-fixing computations performed on a standard laptop computer. No execution-time limit was imposed, but benchmarking computations for standard matrix multiplication were restricted to models whose projection matrix required less than 2 GB of memory.

9. Figure 3C: what do the black dots indicate?

The black dots indicate the fixed reference nucleotides for the core motif in each register. We have added the following clarification to the figure legend:

Black dots mark the reference nucleotide at each position of the core motif, as indicated in panel A.

10. Section on: Application to the Shine-Dalgarno fitness landscape. I understand the authors refer to another work for details, but it would be useful to report quickly a few details on the parameters shown. How were they inferred? Was any regularisation used? How many data were used?

We have expanded the text to state the amount of data used and to describe the inference procedure. The measured data are now quantified as:

This yielded activity measurements for 257,565 of the $4^9 = 262,144$ possible 9-nt sequences.

and the inference procedure is described in the following paragraph:

Recently, Martí-Gómez et al. (2026) extended this landscape to all possible 9-nt sequences using variance component regression, a Bayesian inference method based on Gaussian processes (Zhou et al., 2022). This method estimates the variance attributable to genetic interactions of each order, then uses these estimates to define a prior over landscapes.

Regularization enters through this prior rather than through an explicit penalty term.

11. The sentence: ‘For each peak we define a distribution π that fixes the AGGAG core motif in the corresponding register’: could one report the mathematical formulation of this choice? I think it would be useful to potentially make these findings relatable to the standard gauges in the field, i.e., is this choice similar to a zero-sum gauge or others?

We have revised the main text to provide an explicit mathematical definition of the distribution associated with each register:

Then for each register i , we defined a distribution π that fixes the core motif $u = \text{AGGAG}$ at that register while randomizing the other positions within the 9-nt variable region (Figure 3A). [FOOTNOTE: Formally, $\pi_{i+j-1}^c = x_j^c(u)$ for $1 \leq j \leq 5$, $\pi_l^c = 0.25$ for $l < i$ or $l > i + 4$.]

The gauges used in this example belong to the family of generalized wild-type gauges, and we have revised the text to say so explicitly. The interpretation of the resulting parameters is stated where the gauges are defined, and is then developed for each parameter type in turn as the corresponding results are presented (constant parameters, additive parameters, and pairwise interactions, respectively).

These gauges are examples of the generalized wild-type gauges described by Posfai et al. (2025b), in which a reference sequence is specified at only a subset of positions. Under such a gauge, the parameters describe the average effects of mutations away from the reference sequence, computed over random backgrounds at the remaining positions. We emphasize that, although the resulting parameter values differ across gauges, they are alternative representations of the same underlying model and therefore yield identical predictions.

References

- Weinberger ED (1991). Fourier and Taylor series on fitness landscapes. *Biological Cybernetics*, 65 (5):321–330.
- Poelwijk FJ, Krishna V, Ranganathan R (2016). The Context-Dependence of Mutations: A Linkage of Formalisms. *PLOS Computational Biology*, 12(6):e1004771. Publisher: Public Library of Science.
- Stormo GD (2011). Maximally efficient modeling of DNA sequence motifs at all levels of complexity. *Genetics*, 187(4):1219 – 1224.
- Brookes DH, Aghazadeh A, Listgarten J (2022). On the sparsity of fitness functions and implications for learning. *Proceedings of the National Academy of Sciences*, 119(1):e2109649118.
- Tsui D, Talreja K, Aghazadeh A (2026). Protein fitness landscapes are simpler under evolutionary distributions. *bioRxiv*.
- Faure AJ, Lehner B, Pina VM, Colome CS, Weghorn D (2024). An extension of the walsh-hadamard transform to calculate and model epistasis in genetic landscapes of arbitrary shape and complexity. *PLOS Computational Biology*, 20(5):e1012132.
- Posfai A, McCandlish DM, Kinney JB (2025a). Symmetry, gauge freedoms, and the interpretability of sequence-function relationships. *Physical Review Research*, 7(2):023005.
- Posfai A, Zhou J, McCandlish DM, Kinney JB (2025b). Gauge fixing for sequence-function relationships. *PLOS Computational Biology*, 21(3):e1012818.
- Martí-Gómez C, Zhou J, Chen WC, Stoltzfus A, Kinney JB, McCandlish DM (2026). Inference and Visualization of Complex Genotype–Phenotype Maps. *Molecular Biology and Evolution*, 43 (2):msag023.

- Petti S, Martí-Gómez C, Kinney JB, Zhou J, McCandlish DM (2026). On learning functions over biological sequence space: relating gaussian process priors, regularization, and gauge fixing. *Journal of Mathematical Biology*, 92(5):78.
- Zhou J, Wong MS, Chen WC, Krainer AR, Kinney JB, McCandlish DM (2022). Higher-order epistasis and phenotypic prediction. *Proc. Natl. Acad. Sci. USA*, 119(39).